

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	D.Yaswanth Sri Dana Ganesh	Reg. No.:	192425351
Date:	06/08/2026	Duration:	60 minutes

Code of Conduct

I D.yaswanth Sri Dana Ganesh (192425351) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dana Ganesh

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

ARCHITECTURE DESIGN ANALYSIS REPORT

Satellite Data Visualization Platform for Weather and Climate Pattern Analysis

Introduction

This report presents an Architecture Design Analysis for the proposed Satellite Data Visualization Platform for Weather and Climate Pattern Analysis. The platform is intended to make satellite-derived weather and climate data — currently locked inside complex scientific formats and specialised tools — accessible to a wider range of stakeholders, including meteorologists, researchers, disaster management agencies, policymakers, and students. The report analyses the system's functional and non-functional requirements, selects and justifies a suitable software architecture, presents an architecture diagram, explains the responsibilities and interactions of each component, evaluates the design against key quality attributes, and justifies the architectural decisions made.

1. Analysis of System Requirements

1.1 System Objectives

Based on the problem statement, the system is expected to:

- Ingest and process satellite imagery from public sources such as NASA and NOAA.
- Visualise real-time and historical weather patterns on interactive maps.
- Provide time-series analysis of climate variables such as temperature, rainfall, and cloud cover.
- Automatically detect anomalies such as cyclone formation, droughts, or temperature spikes.
- Allow comparative overlay of multiple data layers (e.g., precipitation vs. vegetation index).
- Support data export for further research use.
- Offer region-specific filtering and a simple, low-barrier web interface.

1.2 Functional Requirements

ID	Functional Requirement
FR1	Retrieve and ingest satellite imagery (infrared, visible, radar) from NASA/NOAA APIs on a scheduled and on-demand basis.
FR2	Normalise heterogeneous satellite data formats (GRIB, NetCDF, HDF5) into a common internal representation.
FR3	Render an interactive map-based visualisation of real-time and historical weather patterns.
FR4	Generate time-series charts for temperature, rainfall, and cloud cover over a user-selected period.
FR5	Detect and flag anomalies such as cyclone formation, drought conditions, or temperature spikes.
FR6	Allow users to overlay and compare multiple data layers on the same map (e.g., precipitation vs. vegetation index).

ID	Functional Requirement
FR7	Allow authenticated users to export processed datasets and visualisations (CSV, image, GeoTIFF).
FR8	Provide region-specific filtering of data by country, state, or custom bounding box.
FR9	Provide a simple, browser-based interface usable without specialist GIS or remote-sensing software.

1.3 Non-Functional Requirements & Quality Attributes

Quality Attribute	Requirement
Scalability	The platform must handle growing volumes of satellite imagery and a growing number of concurrent users (researchers, agencies, students) without redesign.
Performance	Map tiles and chart data must load quickly (sub-second to a few seconds) even when querying large historical time ranges.
Reliability	Ingestion and processing pipelines must recover gracefully from source-API failures or malformed data without data loss.
Availability	The platform should remain available with minimal downtime, particularly during active disaster/extreme-weather events when it is most needed.
Maintainability	Individual processing modules (ingestion, pattern detection, visualisation) must be independently updatable as new data sources or algorithms are added.
Security	Access to raw data, exports, and administrative functions must be authenticated and authorised; data in transit must be encrypted.
Usability	Non-expert users (planners, students, local officials) must be able to use the system with minimal training.
Interoperability	The system must integrate with external, publicly available APIs (NASA, NOAA) and standard geospatial formats.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

A Hybrid Architecture combining Microservices and Event-Driven Architecture (EDA), fronted by an API Gateway and a layered client-facing presentation tier, is proposed for this system.

2.2 Justification

The problem statement describes a system with several independent concerns — data ingestion, format normalisation, pattern/anomaly detection, time-series analytics, layer comparison, export, and visualisation — each with distinct scaling and update needs. A single monolithic or purely layered design would tightly couple these concerns and make it difficult to scale the compute-heavy processing modules independently of the lightweight visualisation front end. The following factors justify the hybrid Microservices + Event-Driven approach:

- Independent scalability: Pattern detection and time-series analytics are compute-intensive and bursty (e.g., during a cyclone event), while the visualisation backend is I/O-bound. Microservices allow each to scale independently.
- Asynchronous, high-volume data flow: Satellite data arrives continuously and in large volumes. An event bus (e.g., Apache Kafka) decouples ingestion from downstream processing, so slow consumers do not block new data from arriving.
- Extensibility: The problem statement explicitly calls for "a scalable foundation for integrating additional satellite data sources and advanced predictive analytics in the future." New data sources or analytics can be added as new services/consumers without touching existing ones.
- Fault isolation: If the pattern-detection service fails, ingestion and map visualisation can continue to function, which is important for reliability during disaster response.
- Technology fit: Different services (imagery normalisation, geospatial analytics, chart generation) can use the most suitable language/library for each task, which is difficult in a monolith.

A pure microservices design without an event bus was considered but rejected because direct service-to-service calls would create tight temporal coupling between the ingestion pipeline and multiple downstream analytics services, reducing reliability during data bursts. A purely layered monolithic architecture was also considered; it is simpler to build initially but does not meet the scalability and extensibility objectives stated in the problem, so it was rejected as the primary style, although a layered structure is retained inside the presentation tier and API gateway for clarity.

3. Software Architecture Diagram

The diagram below illustrates the overall structure of the proposed system, including the external data sources, ingestion layer, event bus, processing microservices, data stores, API gateway, and presentation layer.

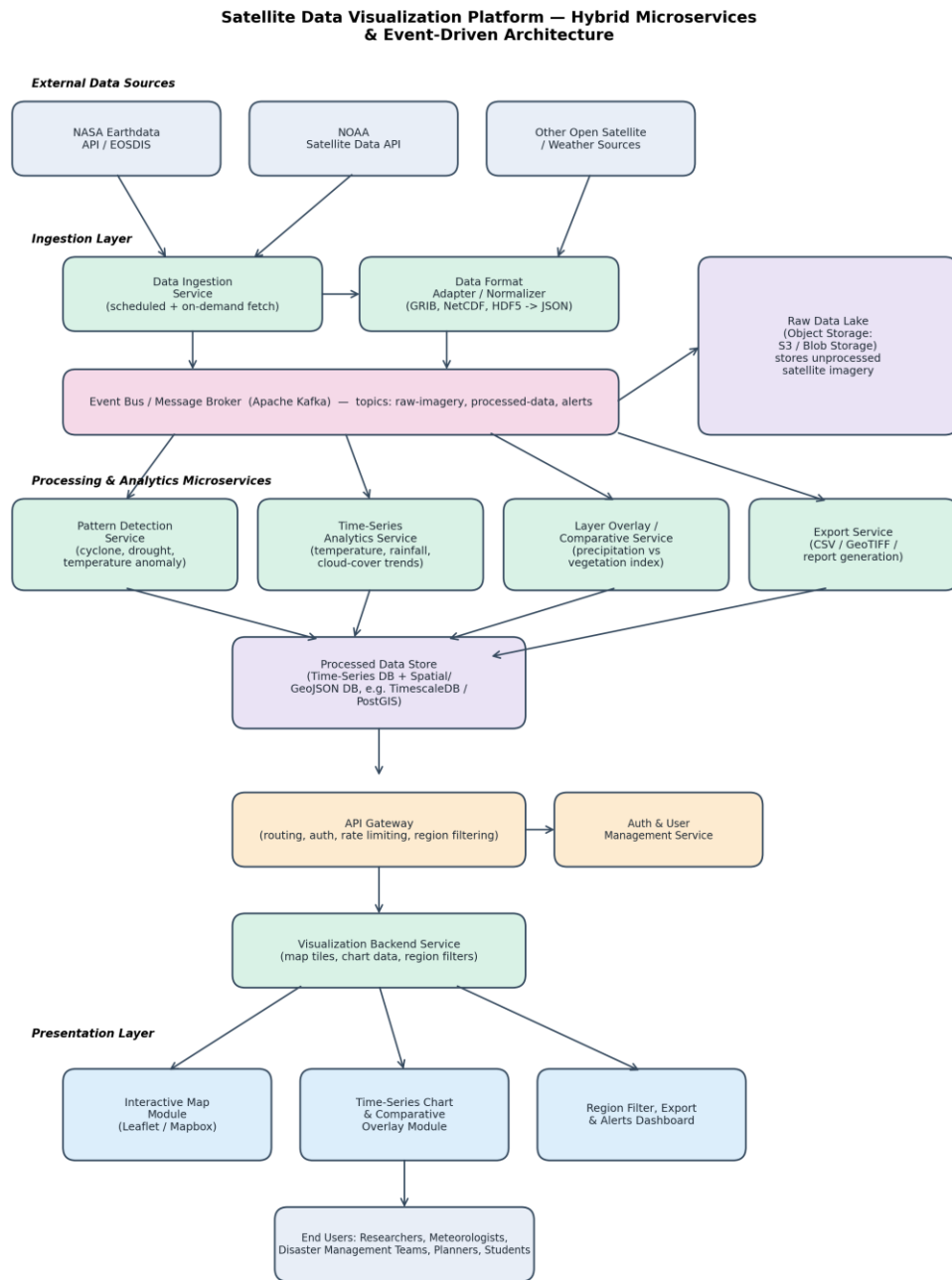


Figure 1: Hybrid Microservices & Event-Driven Architecture for the Satellite Data Visualization Platform

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility
External Data Sources	NASA Earthdata/EOSDIS, NOAA, and other public satellite/weather APIs that provide raw imagery and measurement data.
Data Ingestion Service	Periodically polls and/or receives satellite data from external APIs; handles retries and rate limits; publishes raw data events to the message broker.
Data Format Adapter / Normalizer	Converts heterogeneous scientific formats (GRIB, NetCDF, HDF5) into a common internal schema (e.g., GeoJSON/JSON) for downstream consumption.

Component	Responsibility
Event Bus (Apache Kafka)	Decouples producers and consumers; carries topics such as raw-imagery, processed-data, and alerts, enabling asynchronous, resilient data flow.
Raw Data Lake	Object storage (e.g., S3-compatible) that retains unprocessed satellite imagery for reprocessing, auditing, and future analytics.
Pattern Detection Service	Consumes normalised data to identify anomalies — cyclone formation, drought conditions, temperature spikes — and publishes alerts.
Time-Series Analytics Service	Aggregates historical and real-time data into time-series trends for temperature, rainfall, and cloud cover.
Layer Overlay / Comparative Service	Prepares combined data layers (e.g., precipitation vs. vegetation index) for comparative visualisation.
Export Service	Generates downloadable datasets and reports (CSV, GeoTIFF, images) on user request.
Processed Data Store	A time-series and spatial database (e.g., TimescaleDB/PostGIS) holding analysis-ready data for fast querying by region and time.
API Gateway	Single entry point for client requests; handles routing, authentication delegation, rate limiting, and region-based filtering.
Auth & User Management Service	Manages user accounts, roles (researcher, planner, administrator), and access tokens.
Visualization Backend Service	Serves map tiles, chart data, and filtered query results to the front end, translating stored data into presentation-ready responses.
Presentation Layer (Web Client)	Interactive map module, time-series/comparative chart module, and region-filter/export/alerts dashboard used by end users.

4.2 Component Communication

- The Data Ingestion Service communicates with external APIs over HTTPS/REST and publishes messages to the Event Bus.
- All processing microservices (pattern detection, time-series analytics, layer overlay) subscribe to relevant Kafka topics and consume data asynchronously — they do not call each other directly, which keeps them loosely coupled.
- Processed results are written to the Processed Data Store; the Raw Data Lake retains unprocessed imagery independently.
- The Visualization Backend Service and Export Service query the Processed Data Store synchronously (via internal REST/gRPC) to serve client requests.
- The API Gateway is the single point of contact for the web client, routing requests to the Auth Service and Visualization Backend, and enforcing region-based filtering rules.
- The Presentation Layer communicates with the API Gateway exclusively over HTTPS/REST (and WebSocket for real-time alerts), never accessing internal services directly.

4.3 Overall Workflow

1) Satellite data is fetched from NASA/NOAA by the Ingestion Service and published to the Event Bus. 2) The Format Adapter normalises the data and republishes it. 3) Raw imagery is archived to the Data Lake while normalised events are consumed in parallel by the Pattern Detection, Time-Series Analytics, and Layer Overlay services. 4) Results are persisted to the Processed Data Store, and any detected anomalies raise alert events. 5) When a user interacts with the web client, the request passes through the API Gateway (with authentication) to the Visualization Backend Service, which queries the Processed Data Store and returns map tiles, chart data, or export files to the client.

5. Discussion of Quality Attributes

Scalability: Each microservice can be scaled horizontally and independently based on load — for example, the Pattern Detection Service can be scaled up during active storm events while the Export Service remains at baseline capacity. The Event Bus buffers load spikes so ingestion is never blocked by slow consumers.

Security: The API Gateway centralises authentication and authorisation, so individual services do not need to duplicate security logic. All external and internal traffic uses TLS/HTTPS, and role-based access control restricts who can view, export, or administer data.

Performance: Precomputed, indexed data in the Processed Data Store (time-series and spatial indexes) allows the Visualization Backend to serve map tiles and chart queries quickly, rather than reprocessing raw imagery on every request. Asynchronous processing via the Event Bus keeps ingestion throughput high.

Reliability: Kafka provides message durability and at-least-once delivery, so data is not lost if a downstream service is temporarily unavailable. Failed ingestion or processing tasks can be retried without affecting other pipelines.

Maintainability: Because responsibilities are split into small, independently deployable services, new algorithms (e.g., an improved cyclone-detection model) or new data sources can be added or updated without redeploying the entire system.

Availability: Stateless services behind the API Gateway can be replicated across multiple instances/nodes, and the decoupled design means a fault in one analytics service does not bring down ingestion or visualisation — important for continuous availability during disaster response.

6. Justification of Architectural Decisions

6.1 Rationale

The hybrid Microservices and Event-Driven Architecture was chosen because the problem inherently consists of loosely related processing stages operating on a continuous stream of large-volume satellite data, serving a diverse set of stakeholders with very different usage patterns (occasional public users browsing maps versus researchers running heavy analytical queries). This architecture directly maps system objectives to the quality attributes that matter most for the domain: scalability under bursty load, reliability during high-stakes disaster events, and maintainability as new data sources and analytics are added over time.

6.2 Trade-offs Considered

- Complexity vs. flexibility: Microservices introduce operational complexity (service discovery, monitoring, distributed debugging) compared to a monolith, but this is accepted in exchange for independent scalability and fault isolation, which are critical given the bursty, high-volume nature of satellite data.
- Eventual consistency vs. simplicity: The event-driven design means processed data may lag slightly behind raw ingestion (eventual consistency). This is an acceptable trade-off because climate/weather analytics do not require strict real-time consistency, unlike, for example, a financial transaction system.
- Infrastructure cost vs. resilience: Running a message broker, multiple databases, and several services increases infrastructure and DevOps overhead compared to a simpler client-server design, but this cost is justified by the reliability and scalability needs identified in the requirements analysis.

6.3 Support for Future Enhancements

The architecture directly supports the stated goal of a "scalable foundation for integrating additional satellite data sources and advanced predictive analytics." New data sources can be added as additional producers to the Event Bus without modifying existing services. New analytics (e.g., machine-learning-based forecasting) can be added as new Kafka consumers reading from the same processed-data topics. The API Gateway allows new client applications (e.g., a future mobile app) to reuse all existing backend services without change, supporting long-term system integration and maintainability.

7. Conclusion

The proposed Hybrid Microservices and Event-Driven Architecture addresses the core challenge in the problem statement: transforming complex, high-volume, multi-source satellite data into an accessible, interactive platform for a broad range of non-expert and expert stakeholders. By decoupling ingestion, processing, storage, and presentation, the design achieves the scalability, reliability, and maintainability required, while remaining extensible for future data sources and predictive analytics capabilities.